

Forward progress delegation for executors

Document #: P1898R1
Date: 2020-05-16
Project: Programming Language C++
SG1
Reply-to: Lee Howes
<lwh@fb.com>

§ 1 Changelog

§ 1.1 Differences between R0 and R1

- Tidying of introduction.
- Updated wording to more clearly make changes necessary to P1898R3.

§ 2 Introduction

This design enables delegation of forward progress in asynchronous algorithms and enables well-defined propagation of execution resources through an asynchronous work chain. This work is based in part on a discussion of the `DeferredExecutor` that works with [folly's `SemiFuture`](#) and how `folly`'s coroutines propagate executors as well to ensure that work runs on predictable underlying execution resources.

In this paper executor refers to the general concept as expressed in the title of [\[P0443R13\]](#) and the long standing executors effort. The concrete concepts used may satisfy executor, scheduler or sender depending on the context.

Building on the definitions in [\[P0443R13\]](#) and [\[P1897\]](#), we start with the definition of a receiver we propose adding a concept `scheduler_provider` that allows a sender to require that a receiver passed to it is able to provide an executor, or more accurately a scheduler in the language of [\[P0443R13\]](#). This concept offers a `get_scheduler` customization point that allows a scheduler to be propagated from downstream to upstream asynchronous tasks, or alternatively from upstream to downstream. A given scheduler, and therefore any sender created off that scheduler may require the availability of such a customization on receiverss passed to the `submit` operation.

A given sender may delegate any tasks it is unable to perform to the scheduler it acquires from the downstream `Callback`. A given sender alternatively might always reschedule work to an underlying scheduler, and in situations where the chosen scheduler is

ambiguous, for example where the output work runs from a `when_all` operation, that sender might require that downstream work provide a scheduler for it to reschedule onto to remove nondeterminism in where work runs.

A `sync_wait` operation, as defined in [P1897], encapsulates a scheduler that wraps a context owned by and executed on the calling thread. This will make progress and as such can act as the executor-of-last-resort when the C++20 guarantee is required in blocking code.

We also propose that a set of definitions for forward progress of asynchronous graph nodes be provided.

§ 3 Motivation and Scope

§ 3.1 Forward progress delegation in C++20.

The standard parallel algorithms support weak forward progress guarantees. For example, with the code:

```
vector<int> vec{1, 2, 3};
vector<int> out(vec.size(), 0);
transform(
    std::execution::par,
    begin(vec),
    end(vec),
    begin(out),
    [](int i){return i+1;} // s0
); // s1
```

Allows the runtime to assume that the statement `s0` is safe to execute concurrently for some set of agents, and makes the guarantee that once started each instance of `s0` will run to completion, independent of whatever else is happening in the system.

However, the same is true of statement `s1`. Once started, `s1` must also run to completion. As a result, while the runtime makes no guarantee about instances of `s0` the collection of such instances must complete - and therefore every instance must run eventually.

In practice, on shared resources, other work in the system might occupy every available thread. To guarantee that all `s0`s are able to run eventually, we might need to create three threads to achieve this - and in the extreme this might not be possible.

We therefore fall back to a design where the calling thread may run the remaining work. We know the caller is making progress. If no thread ever becomes available to run the work the caller might iterate through all instances of `s0` serially.

§ 3.2 Forward progress delegation in an asynchronous world

Unfortunately, if transform is made asynchronous, which we will call `range_transform` below, then we have a challenge. Take a result-by-return formulation of async transform as an example.

In either an imagined coroutine formulation:

```
awaitable<vector<int>> doWork(vector<int> vec) {  
    co_return co_await range_transform(  
        vec, [](int i) {return i+1;});  
}
```

Or a declarative formulation as in [P1897]:

```
sender-of<vector<int>> doWork(sender-of<vector<int>> vec) {  
    return std::move(vec) | range_transform([](int i) {return  
i+1;});  
}
```

There is no guaranteed context to delegate work too. We cannot block the caller in either case - the body itself is asynchronous code. We need a more general approach.

§ 4 Examples of use

§ 4.1 Delegation in the folly futures library

In Facebook's folly library we have a limited, but extreme, form of forward progress delegation in the `DeferredExecutor`. As folly has evolved from an eager Futures library and added laziness, and because folly's executors are passed by reference, this is specialized in folly and is a hidden implementation detail of the `SemiFuture` class. The principle would apply the same if it were made public, with appropriate lifetime improvements. The `DeferredExecutor` functions as a mechanism to allow APIs to force the caller to specify work can run, but to also allow them to push some work on to the caller. For example, an API that retrieves network data and then deserializes it need not provide any execution context of its own, it can defer all deserialization work into a context of the caller's choice.

`DeferredExecutor` is a single-slot holder of a callback. It will never run tasks itself, and represents no execution context of its own. The `DeferredExecutor` instance will acquire an execution context when it is passed a downstream executor.

So, for example take this simple chain of work:

```
SemiFuture<int> f1 = doSomethingInterestingEagerly();  
SemiFuture<int> f2 = std::move(f1)  
    .deferValue([](int i){return i+1;}); // s3  
Future<int> f3 = f2.via(&aRealExecutor);
```

Even if `f1` is complete at the point that `s3` is enqueued onto the `SemiFuture`, `f2` will never become ready. The `deferValue` call enqueues work and ensures that there is a

DeferredExecutor on the future, replacing the executor was that completed the work resulting in f1.

When f1 completes it will pass callback s3 to the executor's execute method, and that callback will be held there, with no context to run it on.

With the creation of f3, an executor is attached and is passed to the DeferredExecutor's set_executor method. At that point, if f1 was complete, s3 would be passed through to aRealExecutor. All forward progress here has been delegated from the DeferredExecutor to aRealExecutor.

Note that a blocking operation:

```
SemiFuture<int> f1 = doSomethingInterestingEagerly();
SemiFuture<int> f2 = std::move(f1)
    .deferValue([](int i){return i+1;}); // s3
int result = std::move(f2).get();
```

implicitly carries an executor driven by the caller, which is propagated similarly

A coroutine:

```
int result = co_await std::move(f2);
```

similarly propagates an executor that the coroutine carries with it in the implementation of folly's Task. Unlike in the synchronous example, the coroutine is not blocking the caller, it is simply transferring work from an executor that is unable to perform it to the one the caller is aware of.

§ 4.2 Submitting work to an accelerator

We may have a chain of tasks that we want to run on an accelerator, but that are dependent on some input trigger. For example, in OpenCL we might submit a sequence of tasks to a queue, such that the first one is dependent on an event triggered by the host.

```
host_event | gpu_task | gpu_task | gpu_task
```

We do not know when the host event completes. It may be ready immediately. When do we submit the gpu work? There are heuristics for doing this:

- The queue might be being consumed constantly by a hardware thread.
- There may be a background driver thread pulling from the queue.
- Some submit operations may cause work to happen.

The last one is unstable if we do not know for sure that more work will arrive. Some hardware queuing models, such as HSA, require an explicit action to mark a set of tasks as valid. If we are building that queue from a weakly parallel agent, or if we want to delay passing the queue to the device until host_event is ready but we do not know whether the task completing host_event has a strong enough forward progress guarantee to safely call

into a driver to trigger that queue copy we can instead structure this so that we require an executor to be specified. In that case the chain of tasks may require an executor to be attached at the end of the chain: a task may be enqueued onto that executor, that flushes the queue. This may remove the need for background threads.

§ 4.3 Supporting overflow from a fixed size thread pool

This is the traditional case for the parallel algorithms as specified in C++20. If for whatever reason a thread pool backing an executor is unable to take more work, for example it has stopped processing and not freed its input queue, then it may delegate work to the downstream executor.

Clearly, there are challenges with implementing this and making a true guarantee in all cases. What I believe is clear is that this has to be implemented in the executor - a queue of work passed to the executor is no longer owned by the algorithm. The algorithm is in no position to change where this work is scheduled once the executor owns it. Therefore the most appropriate solution is to make available a valid downstream executor to the upstream executor and allow one to delegate to the other.

§ 5 Generalising

Let us say that a given asynchronous algorithm promises some forward progress guarantee that depends on its executor/scheduler¹.

Note that in the examples below *sender-of* refers to a sender that will call `set_value` on a receiver passed to its `connect` operation with type `T`, and exists for exposition only.

So in a work chain (using `|` to represent a chaining of work as in C++20's Ranges):

```
sender-of<vector<int>> doWork(sender-of<vector<int>> vec) {  
    return std::move(vec) | on(DeferredExecutor{}) |  
    range_transform([](int i) {return i+1;});  
}
```

`DeferredExecutor` is a executor known to make no forward progress guarantee for its algorithms. Indeed, if implemented as in folly, quite the opposite is true. It is guaranteed to make no progress. Note that this is laziness of runtime, but is independent of laziness of submission of a work chain.

As a result, the returned sender also makes no progress. We may want to be able to query that the resulting sender makes no progress, but we leave that to later. What it certainly means is that if we add a node to this chain, it must be able to provide an executor to this sender. The requirement to be provided with a executor can be enforced at compile time.

The mechanism to achieve this is that the `submit` method on the returned sender requires that the receiver passed to it offers an executor. At worst, this executor might also be

deferred, but it is a requirement that at the point this work is made concrete, the `DeferredExecutor` has an executor available to delegate to.

Now let's say we explicitly provide an executor with a strong guarantee. A concurrent executor like `NewThreadExecutor`:

```
sender-of<vector<int>> doWork(sender-of<vector<int>> vec) {  
    return std::move(vec) |  
        on(DeferredExecutor{}) |  
        range_transform([](int i) {return i+1;}) | // c2  
        on(NewThreadExecutor{});  
}
```

Now the algorithm `c2` is guaranteed to run on the `NewThreadExecutor`. It is guaranteed to make concurrent progress, which is enough to ensure that the `range_transform` starts and completes in reasonable time.

Note that this also provides an opportunity for an intermediate state. If instead of a `DeferredExecutor` we had a `BoundedQueueExecutor` with the obvious meaning and a queue size of 1:

```
sender-of<vector<int>> doWork(sender-of<vector<int>> vec) {  
    return std::move(vec) |  
        on(BoundedQueueExecutor{1}) |  
        range_transform([](int i) {return i+1;}) | // c2  
        on(NewThreadExecutor{});  
}
```

`range_transform` will launch a task per element of `vec` and `vec` is larger than 1. As a result, the second task may be rejected because of the queue bound. One option is to propagate a failure error downstream, such that the `range_transform` would fail with an error. However, if `BoundedQueueExecutor` can make use of the scheduler provided by the downstream receiver, it can instead delegate the work, ensuring that even though it hit a limit internally, the second scheduler will complete the work.

Any given executor should declare in its specification if it will delegate or not. If no delegate executor is provided, for example we detach the work as below:

```
void doWork(sender-of<vector<int>> vec) {  
    std::move(vec) |  
        on(BoundedQueueExecutor{1}) |  
        range_transform([](int i) {return i+1;}) |  
        handle_error([](Ex&& ex) {Handle the error}) |  
        detach();  
}
```

then no delegating executor is available to delegate to, and we might instead produce the error which can be handled appropriately.

Finally, note that this is more flexible than throwing an exception on enqueue - the `submit` operation on the sender provided by both of the above instances of

BoundedQueueExecutor can always succeed, removing the risk of trying to handle an enqueue failure from the caller. One will propagate an error inline, the other gives the executor the decision of whether to propagate work or not. That allows it to make the decision to delegate after a *successful* enqueue - for example when the queue has accepted work but the underlying execution context realizes some property cannot be satisfied.

§ 5.1 Synchronous operations

Providing the same guarantee as the synchronous transform:

```
vector<int> vec{1, 2, 3};
vector<int> out(vec.size(), 0);
std::transform(
    std::execution::par,
    begin(vec),
    end(vec),
    begin(out),
    [](int i){return i+1;} // s0
); // s1
```

May be achieved by something like:

```
vector<int> doWork(sender-of<vector<int>> vec) {
    vector<int> output;
    ManualExecutor m;
    std::move(vec) |
        bulk_transform(std::execution::par, [](int i) {return i+1;}) |
        transform([&output](vector<int> v){output = std::move(v);}) |
        on(m);
    m.drain();
    return output;
}
```

That is that the forward progress of the transform, if it is unable to satisfy the requirement, may delegate to the ManualExecutor. This is an executor that may be driven by the calling thread, in this case by the drain method. So we support the delegation of work onto the calling thread by providing an executor to delegate to, but donating the current thread's forward progress to that executor until all the work is complete.

This would of course be wrapped in a wait algorithm in practice, as proposed in [P1897]:

```
vector<int> doWork(sender-of<vector<int>> vec) {
    return sync_wait(std::move(vec) |
        bulk_transform(std::execution::par, [](int i) {return i+1;}));
}
```

sync_wait applied to an asynchronous bulk operation maintains the forward progress delegating behaviour we offer in the blocking version of the algorithm in C++20.

§ 6 Impact on the Standard

- Add `scheduler_provider` concept.
- Add `get_scheduler` CPO.
- Additional wording to the `sync_wait` CPO specifying that it will pass an `scheduler_provider` to the connect operation on the passed sender.
- Additional wording to the `on` CPO specifying that it will pass an `scheduler_provider` to the connect operation on the passed sender.
- Additional wording for other algorithms specifying that they pass through the `get_scheduler` CPO, making their own receivers also `scheduler_providers`.

§ 6.1 Concept `scheduler_provider`

§ 6.1.1 Summary

A concept for receivers that may provide a scheduler upstream. May be used to overload connect algorithm, to allow delegation in the presence of a downstream scheduler. May be used to restrict the connect operation on a sender to require an `scheduler_provider`.

§ 6.1.2 Wording

```
template<class R>
concept scheduler_provider =
    requires(SP&& sp) {
        { execution::get_scheduler((SP&&) sp) -> scheduler } noexcept;
    };
```

§ 6.2 `execution::get_scheduler`

§ 6.2.1 Summary

When applied to a `scheduler_provider`, if supported, will return a scheduler that the `scheduler_provider` expects to be able to run delegated work safely, and that callers may assume they are able to delegate work to.

§ 6.2.2 Wording

The name `execution::get_scheduler` denotes a customization point object. For some subexpression `sp`, let `SP` be a type such that `decltype((sp))` is `SP`. The expression `execution::get_scheduler(sp)` is expression-equivalent to:

- `sp.get_scheduler()` if that expression is valid and if the expression return type satisfies `scheduler`
- Otherwise, `get_scheduler(sp)` if that expression is valid, if the expression return type satisfies `scheduler` with overload resolution performed in a context that includes the declaration

```
void get_scheduler() = delete;
```

and that does not include a declaration of `execution::get_scheduler`. * All work scheduled on a scheduler returned by `execution::get_scheduler(sp)` must complete before the destructor of `sp`. **Note:** This is because the execution context underlying the scheduler may have a lifetime bounded by the lifetime of `sp`.

§ 6.3 `execution::sync_wait` modifications

§ 6.3.1 Summary

Modifications to `sync_wait` as specified in [P1897]. Rather than simply passing a receiver to the passed sender, `sync_wait` constructs an execution context on the blocked thread, such that the blocked thread drives that executor and executed enqueued work. A scheduler representing that execution context will be made available from the receiver passed to connect on the sender, thus making that receiver a `scheduler_provider`.

§ 6.3.2 Wording to modify final otherwise clause

- Otherwise:
 - Constructs an execution context owned by the calling thread such that tasks enqueued into that context will be processed by the calling thread.
 - Constructs a scheduler, `sch` representing that execution context.
 - Constructs a receiver, `r` over an implementation-defined synchronization primitive and passes that callback to `execution::connect(s, r)` returning an `operation_state os` such that:
 - `r` satisfies `scheduler_provider`.
 - `execution::get_scheduler(r)` will return `sch`.
 - If the operation completes by calling `set_value(r, t)` then `sync_wait()` will return a value, `x`, of type `remove_cvref_t<decltype(t)>`.
 - If the operation completes by calling `set_value(r)` then `sync_wait()` will return `void`.
 - If the operation completes by calling `set_error(r, e)` then `sync_wait()` calls `std::rethrow_exception(e)` if `decltype(e)` is `std::exception_ptr` or `throw e`; otherwise.
 - If the operation completes by calling `set_done(r)` then `sync_wait()` will call `std::terminate`.
 - Calls `execution::start(os)`.
 - Waits on the synchronization primitive to block on completion of `s`, driving the execution context as necessary until `set_value`, `set_error` or `set_done` is called on `r`.

The thread that invokes `sync_wait` will block with forward progress delegation on completion of the work where supported by the implementation of `s`.

§ 6.4 **execution::on modifications**

§ 6.4.1 **Summary**

In addition to transitioning execution from one execution context to another, on also changes the scheduler that is propagated upstream through scheduler_providers.

§ 6.4.2 **Wording to modify otherwise clause**

Add the clauses: * If output_receiver satisfies scheduler_provider then
execution::get_scheduler(r2) returns the result of
execution::get_scheduler(output_receiver). * r satisfies scheduler_provider and
execution::get_scheduler(r) returns sch.

§ 6.5 **Delegation through asynchronous algorithms**

§ 6.5.1 **Summary**

Most algorithms do not provide executors. These algorithms should avoid breaking the forward progress delegation chain and should allow executors to propagate from receiver to sender in a chain.

§ 6.5.2 **Wording**

To execution::when_all add the text: * If output_receiver satisfies scheduler_provider, execution::get_scheduler(ri) for all ri returns the result of execution::get_scheduler(output_receiver).

To execution::transform, execution::handle_error and execution::let add the text: * If output_receiver satisfies scheduler_provider, execution::get_scheduler(r) returns the result of execution::get_scheduler(output_receiver).

A similar modification should be applied to bulk_execute once the semantics of the returned sender are clear.

§ 7 **Future Work: adding algorithm-level forward progress definitions**

We have definitions in the standard for forward progress guarantees for threads of execution. These apply to the workers in the parallel algorithms fairly clearly.

It is not clear:

- Whether it is safe to call executor, scheduler or sender operations under weak forward progress.

- This will be important for launching nested work.
- It may be important simply for having one worker trigger the next.
- Whether it is safe to build a chain of work in a weak agent.
- What the forward progress guarantee may be of the completion of a parallel algorithm.

All of these are relatively well-defined for synchronous algorithms as they stand. These are blocking algorithms.

Therefore

```
vector<int> vec{1, 2, 3};
vector<int> out(vec.size(), 0);
transform(
    std::execution::par_unseq,
    begin(vec),
    end(vec),
    begin(out),
    [](int i){
        vector<int> innervec{1, 2, 3};
        vector<int> innerout(vec.size(), 0);
        transform(
            std::execution::par_unseq,
            begin(innervec),
            end(innervec),
            begin(innerout),
            [](int i){
                return i+1;
            });
        return innerout[0];
    });
```

is not valid.

However, it is not clear that something like

```
vector<int> vec{1, 2, 3};
vector<int> out(vec.size(), 0);
transform(
    std::execution::par_unseq,
    begin(vec),
    end(vec),
    begin(out),
    [](int i){
        auto s = makeReadySender(0);
        auto resultSender = std::execution::transform(
            std::move(s)
            [](int i){
                return i+1;
            });
        std::execution::submit(
            std::move(resultSender),
```

```
    NullCallback{});  
    return 0;  
});
```

is not valid.

This code is entirely inline. It is blocking, but trivially so because no executor or scheduler is involved. Can we make a claim that building and running this operation satisfies `par_unseq`'s guarantee? Could we if it had a scheduler involved - would `schedule`, `submit` or `execute` satisfy the guarantee?

We believe that we need to start to think through these problems. At a minimum, `submit` on a sender may have to make a statement about the guarantee it makes. senders may then propagate that guarantee, and that of the upstream sender, down a chain.

This is food for thought. We don't have a concrete design in mind, but would like us as a group to be thinking about how asynchronous algorithms and the forward progress definitions interact in general.

§ 8 References

[P0443R13] 2020. A Unified Executors Proposal for C++.

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p0443R13.html>

[P1897] 2020. Towards C++23 executors: A proposal for an initial set of algorithms.

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1897r3.pdf>

-
1. Potentially also on the execution property passed but we leave that out here for simplicity.↩